

마이그레이션 명세서 — 소켓 데이터 → 화면 드로잉 파이프라인

목차

- 문서 목적
- 0. 전체 파이프라인 개요
- 1. 와이어 프로토콜 — 패킷 헤더 `_axisH`
 - `msgK` (메시지 종류) — `h/axis.h:44~70` (전수 확인, 2026-07-31 갱신)
 - `auxs` 부가상태 (확인된 것)
- 2. 패킷 조각 재조립 규칙 — `CWorks::OnStream`
- 3. 창(작업영역) 식별 규칙 — `key / type`
 - `key` — 작업영역 슬롯 식별자 (한 바이트 범위)
 - `type` — 화면 종류 + 표시방식 비트마스크 (한 바이트, 상하위 니블 분리)
 - `unit` — 작업영역 안의 화면(스크린) 식별자 (2026-07-30 확인)
- 4. 화면(맵) 로드/조립 규칙 — `CScreen::Parse()`
 - 4.1 서브맵 임베딩 규칙 (`FM_OBJECT`)
 - 4.2 그리드 특수역할 마커 — `vals[2]` 의 `$$ / $? / $*` (`Screen.cpp:372-385`, 2026-08-14 확인)
- 5. 컨트롤은 실제 윈도우가 아님 — 가상 컨트롤 구조
- 6. 입력 이벤트 → 원시 메시지 후킹 규칙 — 윈도우 서브클래스
 - `CallProc` 이 처리하는 메시지 전체 목록 (27종, `Event.cpp` 실측)
- 7. 이벤트 → 스크립트 실행 규칙
 - 7.1 프로시저 명명 규칙
 - 7.2 실행 게이팅 — `IsAvailable`
 - 7.3 실측 확인된 사실 — 서브맵도 독립적으로 이벤트를 가짐
- 8. 실시간 시세(RTM) 매칭 규칙 — 요약
- 8.5. 데이터 처리부 상세 — `CStream` (`Stream.cpp`)
 - `msgK` 종류 — `OutStream` 까지 도달하는 것만 (TR 요청-응답 계열 하위 집합, `CStream::OutStream` 실측)
 - 필드 단위 파싱 알고리즘 — `SetDataNRM` (`Stream.cpp:1474`, `msgK_AXIS` 의 기본 포맷)
 - 인밴드 제어코드 (FCC/RCC/SCC) — 2026-07-26 정정: 색상 아니라 "속성 제어"
- 8.6. 그리드 셀 데이터 파싱 — `SetCells / SetTable` (`Stream.cpp`)
 - `SetCells` (`Stream.cpp:2444`) — 일반 그리드
 - `SetTable` (`Stream.cpp:2647`) — `GO_TABLE` 방식, 인밴드 제어코드 처리 위치
 - 사용자 정의 컬럼 변형 — `SetCells2 / SetTable2`
- 8.7. 레코드 헤더(`SetDataH`) — 원장(Ledger) 블록의 실제 위치 (2026-07-26 실측 확인)
 - 원장 헤더 구조체 `_ledgerH` — 금융권 표준 "공통전문헤더" 포맷 (`h/ledger.h`)
- 8.8. 송신측 TCP write 프레이밍 — 한 번의 write에 여러 `_axisH` 메시지가 묶일 수 있음 (2026-07-30 실측 확인)
- 8.9. 페이로드 암호화/복호화 — `CGuard::Xecure` (2026-07-31 실측 확인)
 - 확정된 규칙
 - 개발용 진단 스위치 — `NOENC.TXT`
 - 로그 대응표
- 8.10. IBKSCollector OOP 요청 페이로드 문법 — `pooppoop / GOOPHOOP` 계열 (2026-08-14 확인)
 - 배경 — 이 절이 다루는 범위

- 1) 단일값(비그리드) 조회 — 특수문자 없음
- 2) 그리드(다중 레코드) 조회 — \$ 마커
- 3) 캔들/시계열 조회 — ? 마커
- 4) 공통 요약 — 세 문법을 관통하는 규칙
- 5) 확인 범위와 다음 단계
- 8.11. 그리드 실시간 갱신 3종 마커 — \$? / \$\$ / \$* (2026-08-17 확인)
 - 8.11.1. \$? — CScreen::ScrollRTM (순수 삽입, 매칭 없음)
 - 8.11.2. \$\$ — CScreen::OnNotice 후반부 (키매칭 upsert/삭제)
 - 8.11.3. \$* — _anmH 라는 완전히 별도의 소켓 채널 (신규 발견)
- 8.12. Wizard 자체 OOP 왕복 — GetDataOOP / GetDataOOP2 (요청) + SetDataOOP (응답) (2026-08-18 확인)
 - 배경 — 8.10절(IBKSCConnector)과는 별개인, 두 번째 OOP 구현
 - 핵심 발견 — 요청과 응답이 같은 순서표(m_ioR)를 공유해서 정렬을 보장한다 (사용자 가설 확인+정정)
 - 요청 조립 — GetDataOOP (OOP1)/ GetDataOOP2 (OOP2)
 - 응답 파싱 — SetDataOOP (Stream.cpp:1900)
- 9. 다음 조사 대상 (미완료)
- 10. 관련 문서

문서 목적

배경: 연말 HTS 플랫폼 전면 교체 예정 (axis/axwizard 폐기, 새 프로세스/플랫폼으로 이전). 이 문서는 "코드베이스 이해"가 아니라 **새 플랫폼 개발팀이 참고할 수 있는 정확한 사양서**를 목표로 함. 애매한 추정은 배제하고, 실측(로그) 또는 소스 직접 확인으로 검증된 내용만 기록.

범위: 소켓으로 원시 바이트가 들어와서, 화면에 값이 그려지기까지의 전체 파이프라인과 그 안의 모든 규칙.

@docs/WizardArchitecture.md (클래스 계층), @docs/RealtimeCodeIndex_Investigation.md (RTM 매칭 상세)와 상호보완 관계 — 이 문서는 "파이프라인 전체 흐름과 프로토콜"에 집중.

진행상태: 🔍 작성 중 — 소켓 수신→재조립→파싱→필드쓰기(SetDataNRM /그리드 SetCells /원장 SetDataH)까지, 그리고 송수신 암호화(CGuard::Xecure , 8.9절)까지 전부 실측 로그로 검증 완료. SetDataOOP / TAB 류 나머지 포맷, cc_* 플래그 전체 목록, ARM / AUX / DIAL / MAPX msgK 처리 로직은 다음 조사 대상 (9절 참고).

2026-07-25 실측 검증: AxisChaser(패킷캡처 도구)로 원시 바이트를 직접 대조해서 _axisH 구조체 필드 순서 (msgK/stat/auxs/winK/unit)가 실제 와이어 바이트와 1:1로 정확히 일치함을 확인함. 코드분석+로그+원시패킷캡처 3중 검증됨.

2026-07-31 실측 검증: 암호화/복호화(CGuard::Xecure , 8.9절) 전체 흐름과 msgK 전체 값 목록(1절)을 axlog+실제 통신 캡처로 확정. 개발용 NOENC.TXT 스위치로 "서버가 클라이언트의 암호화 선언을 그대로 따라간다"는 사실도 확인.

2026-08-14 확인: IBKSCConnector (OCX) 소스에서 OOP 포맷(trxC=pooppoop/G00PH00P 계열) TR의 **요청 페이로드 조립 문법**을 확정(8.10절 신설) — 단일값/그리드/캔들 세 가지 문법과 \$ / ? 마커, grid_i / grid_o 페이징 구조체까지 소스 코드로 직접 확인. 단, 이건 클라이언트→서버 요청 쪽만 확인된 것이고, Wizard(서버 응답 수신) 쪽의 SetDataOOP 파싱은 여전히 미조사(9절 참고) — 서로 다른 절반.

2026-08-17 확인: 그리드 실시간 갱신 마커 \$? / \$\$ / \$* (8.11절 신설) 전체 추적 완료. 그중 \$* 가 RTM(_rtmH)도 TR(_axisH)도 아닌 **완전히 별도의 소켓 이벤트/헤더(FEV_PUSH , _anmH)를 쓰는 제3의 채널**이라는 걸 신규 발견 — msgK_ARM / msgK_AUX (1절, 미조사 상태였던 것)와의 관계는 아직 미확인.

0. 전체 파이프라인 개요

[서버] TCP 소켓
| (조각날 수 있음, statCON 플래그로 표시)
▼
CWizardCtrl::OnAxis(int type, char* pBytes, int variant) ← 연결 생명주기(FEV_OPEN/FEV_RUN)
|
CWizardCtrl::OnAxis(struct _axisH* axisH, char* pBytes, int nBytes) ← 패킷 1건 진입점
| axisH->winK로 대상 CClient/CDll(works) 탐색, 없으면 서버 지시로 자동 생성 가능
| auxsMAP 플래그면 서버가 화면 전환을 지시할 수 있음
▼
works->OnStream(axisH, pBytes, nBytes) [CWorks::OnStream, Works.cpp:59]
| statCON 비트로 조각 재조립 (m_axisB에 누적)
| 마지막 조각이면:
▼
works->OnAxis(axisH, 재조립된pBytes, nBytes) [가상함수, CClient/CDll이 실제 구현]
| ※ 필드 단위 파싱 상세 - 다음 조사 대상
▼
CScreen 필드에 값 반영 (WriteData 등)
▼
WM_PAINT → CAxisForm::DrawForm → CfmBase::Draw(dc) ← 실제 화면 렌더링

실시간 시세(RTM) 갱신은 별도 경로(CGuard::OnAlert → DoRTM)를 타며 @docs/RealtimeCodeIndex_Investigation.md 에 상세 기록됨 — 이 문서에는 개요만 요약(6절).

1. 와이어 프로토콜 — 패킷 헤더 _axisH

정의: h/axis.h:24

```
struct _axisH {  
    unsigned char msgK;    // 메시지 종류  
    unsigned char stat;    // 상태 (조각전송 플래그 statCON 포함)  
    unsigned char auxs;    // 부가상태 (auxsCLOSE, auxsMAP 등)  
  
    unsigned char winK;    // 대상 창(작업영역) key - axAttach가 배정한 그 key  
    unsigned char unit;    // 유닛 창  
    unsigned char trxK;    // TR key (수신측)  
    unsigned char trxS;    // TR key (송신측)  
  
    char svcN[4];          // 서비스명  
    char trxC[8];          // TR 코드 (화면/맵 코드)  
    char datL[5];          // 뒤따르는 데이터 길이 (문자열)  
};  
// L_axisH = sizeof(struct _axisH) = 24바이트 고정 헤더
```

모든 클라이언트-서버 패킷은 이 24바이트 헤더 + 가변 페이로드 구조입니다. 새 플랫폼에서 소켓을 파싱하려면 이 구조를 그대로 재현해야 함.

msgK (메시지 종류) — h/axis.h:44~70 (전수 확인, 2026-07-31 갱신)

값	상수	의미	처리 경로
0x20	msgK_AXIS	일반 AXIS 메시지 (TR 조회 요청/응답)	OutStream (요청-응답, 8.5절)
0x21	msgK_HTM	HTML 메시지	OutStream
0x22	msgK_TAB	탭 구분 메시지	OutStream
0x24	msgK_SVC	서비스 콜	OutStream
0x25	msgK_APC	승인 콜	OutStream
0x26	msgK_CTRL	컨트롤(FM_CONTROL) 데이터 직접 갱신	OutStream
0x27	msgK_UPF	파일 업로드	OutStream
0x28	msgK_DNF	파일 다운로드	OutStream
0x30	msgK_RSM	리소스 요청 (맵 파일 등, CGuard::RequestMAPs 가 씀)	CGuard::OnRsm (OutStream 안 탐)
0x40	msgK_RTM	실시간 시세 데이터	CGuard::OnAlert → DoRTM (RTM 전용 경로, @docs/RealtimeCodeIndex_Investigation.md)
0x50	msgK_MAPX	맵(화면) 전환 지시	미조사
0x80	msgK_ENC	암호화 키 데이터	CWizardCtrl::OnXecure
0x81	msgK_XCA	인증서(Certify) 데이터	CWizardCtrl::OnXecure
0x82	msgK_SIGN	로그인/로그아웃(sign on/off)	OnStream (OutStream 이 아니라 별도 스트림 진입점)
0x83	msgK_SIGNx	로그인/로그아웃(인증서 로그인)	OnStream
0x90	msgK_TICK	"tick pane(notice...)" — 서버가 임의 시점에 미는 브로드캐스트 알림 (체결통보류). TR 요청-응답이 아니라 요청 없이 도착	CGuard::OnNotice → CScreen::OnNotice (\$\$ 마킹 그리드, 2026-07-31 확인)
0x91	msgK_POP	모달리스 다이얼로그(ASCII 컨트롤 데이터)	OnFire(FEV_AXIS, MAKEWPARAM(alarmPAN,...))
0x92	msgK_ARM	알람 메시지	미조사
0x93	msgK_AUX	AUX 실시간 메시지	미조사
0x94	msgK_DIAL	확인 다이얼로그(axisH.winK 가 세부 다이얼로그 타입)	미조사
0x99	msgK_ERR	에러 메시지	OutStream

중요 — 두 계층의 msgK 분기가 있다. CWizardCtrl::OnRead (WizardCtrl.cpp:719-768)가 소켓에서 막 재조립한 모든 프레임에 대해 **1차로** msgK 를 검사하고, AXIS / TAB / HTM / SVC / APC / CTRL / UPF / DNF / ERR 계열만 CWizardCtrl::OnAxis 를 거쳐 CStream::OutStream (8.5절 표)까지 도달한다. SIGN / SIGNx / RSM / ENC / XCA / TICK / POP 등은 **OutStream 을 아예 타지 않고** OnRead 의 switch 안에서 곧바로 별도 함수로 갈라진다 — 즉 8.5절의 OutStream 표는 "TR 요청-응답 계열"만 다루는 하위 집합

이고, 이 표가 실제 전체 msgK 공간이다. 새 플랫폼의 소켓 디스패처는 이 두 계층 구조(전체 msgK 1차 분기 → 그중 TR 계열만 2차로 요청/응답 매칭)를 그대로 반영해야 한다.

auxs 부가상태 (확인된 것)

플래그	의미	근거
auxsCLOSE	이 창을 닫으라는 서버 지시	WizardCtrl.cpp:1098
auxsMAP	서버가 화면(맵) 전환을 지시 — works->Attach(mapN, true) 트리거	WizardCtrl.cpp:1104-1109
auxsFDS	FDS(시세?) 값 포함 여부 — Guard.cpp 의 Write / Invoke 에서 암호화 분기와 함께 사용	Guard.cpp 다수

2. 패킷 조각 재조립 규칙 — CWorks::OnStream

근거: Works.cpp:59-94

서버는 큰 응답을 여러 TCP 조각으로 나눠 보낼 수 있습니다. axisH.stat 의 statCON 비트가 "뒤에 이어지는 조각이 더 있다"를 뜻합니다.

```
void CWorks::OnStream(_axisH *axisH, char *pBytes, int nBytes)
{
    // 1) 누적 버퍼(m_axisB)에 계속 이어붙임
    //   - 최초 호출: 헤더(L_axisH바이트) + 첫 페이로드
    //   - 이후 호출: 페이로드만 계속 append
    ...
    if (!(axisH->stat & statCON)) // 2) "더 이상 조각 없음" = 완성
    {
        // 3) 메시지 루프를 펌핑하며 대기 (S_ING 상태 플래그가 꺼질 때까지)
        // 4) 재조립된 완전한 메시지를 가상함수 OnAxis로 전달
        OnAxis((struct _axisH*)m_axisB, &m_axisB[L_axisH], m_axisL-L_axisH);
        m_axisL = 0;
        delete [] m_axisB;
    }
}
```

새 플랫폼 구현 시 필수 규칙:

- 헤더는 첫 조각에만 붙어있고, 후속 조각은 순수 페이로드만 옴 (재조립 시 헤더 중복 주의)
- statCON 비트가 켜져있는 동안은 절대 파싱 시도하지 말고 버퍼링만 할 것
- 재조립 도중 메시지 루프를 계속 펌핑함 (UI 프리징 방지 목적으로 보임) — 새 플랫폼이 비동기/논블로킹이면 이 부분은 불필요할 수 있음

3. 창(작업영역) 식별 규칙 — key / type

출처: Wizard/Guard.cpp CGuard::Attach, h/axis.h, h/axisfire.h

key — 작업영역 슬롯 식별자 (한 바이트 범위)

```
winK_NORM = 0x20 // 일반창 시작
winK_POPUP = 0x80 // 팝업창 시작
winK_END = 0xff
```

호출자가 WK_NORM (0x20)/ WK_POPUP (0x80)을 넘기면 "빈 슬롯 아무거나 배정해줘" 요청이고, CGuard::Attach 가 m_major 배열을 순회해 실제 빈 key를 찾아 배정. 구체적 정수를 넘기면 "이 key로 붙여달라" 지정 요청.

_axisH.winK 가 바로 이 key와 동일한 값 — 서버가 보내는 패킷도 이 key로 어느 창을 대상으로 하는지 지정합니다.

type — 화면 종류 + 표시방식 비트마스크 (한 바이트, 상하위 니블 분리)

하위 니블 (0x0F, "종류"):

```
vtypeERR=0x00(에러) vtypeNRM=0x01(일반맵→CClient) vtypeVBX=0x02(레거시,미지원)
vtypeDLL=0x03(DLL→CD11) vtypeGRX=0x04(그래픽) vtypeHTM=0x05(HTML)
```

상위 니블 (0xF0=vtypeMSK, "표시방식"):

```
vtypeSCR=0x10(스크롤형) vtypeFIX=0x20(고정형) vtypeRSZ=0x30(리사이즈-고정)
vtypeWND=0x40(정적창) vtypeFEX=0x50(고정-비스크롤) vtypeREX=0x60(리사이즈-고정 vR)
```

CGuard::Attach 의 switch (type & ~vtypeMSK) 로 하위니블만 보고 cclient (vtypeNRM) 또는 cd11 (vtypeDLL/vtypeGRX) 생성, vtypeVBX 는 명시적으로 거부(WK_NONE 리턴).

실측 예시(2026-07-25 로그): type=97 (0x61) = vtypeREX (0x60,상위) + vtypeNRM (0x01,하위) → 일반 맵 화면, 리사이즈-고정형.

unit — 작업영역 안의 화면(스크린) 식별자 (2026-07-30 확인)

winK 이 작업영역(창) 하나를 식별한다면, 그 작업영역 안에 대표화면 + 임베디드 서브맵(FM_OBJECT , 4.1절 참고)이 여러 개 동시에 존재할 수 있고, 이들도 각자 독립적으로 TR을 송수신한다. 이걸 구분하는 필드가 _axisH.unit 이다.

h/axis.h :

```
#define unitMAIN 0x00 // 대표(메인) 화면 -- trxC에 MAP명이 실림, MAP 전환 가능
#define unitSUB 0xff // 아직 정수 키가 없어 trxC 문자열로 화면을 찾아야 하는 경우
// 그 외 값 = 그 화면의 실제 unit window key
```

Stream.cpp::GetScreen 의 실제 매칭 로직: if (axisH->unit == screen->m_key) return true; — 즉 unit 은 CScreen::m_key 를 그대로 실어 보낸 값이다. CClient::SetAtScreen (대표화면/서브맵 임베딩 공용, Client.cpp:735)이 key == -1 로 호출되면 for (key=0; key<m_magic; key++) 로 빈 슬롯을 찾아 순번을 배정한다 — 대표화면은 항상 0(unitMAIN), 서브맵들은 배정 순서대로 1, 2, 3...을 받는다.

실측 예시(2026-07-29~30 로그, IB120800 이 대표화면, IB120810 이 그 안의 서브맵):

```
[0-MakeStream-send] ----- map=IB120810 tr=pooppoop ----- winK=35 unit=1 <- 서브맵(m_key=1)
[0-MakeStream-send] ----- map=IB120800 tr=piboPBxQ ----- winK=35 unit=0 <- 대표화면(m_key=0=unitMAIN)
```

주의 — unit 은 msgK 에 따라 인코딩이 다르다. 위 규칙(`unit == screen->m_key`)은 `msgK_AXIS / msgK_TAB` (일반 TR 송수신)에만 적용된다. `msgK_CTRL` (0x26, FM_CONTROL 필드 직접 갱신)은 전혀 다른 조회 경로 (`CStream::GetScreen(screen, axisH, index, ukey)` 오버로드)를 타며, 내부적으로 `CClient::GetScreenKey(axisH->unit, index, ukey)` 가 `CClient::m_keys` 라는 별도 록업테이블에서 unit 한 바이트에 패킹된 (화면키+컨트롤인덱스+ukey) 조합을 풀어낸다(`Client.cpp:3769`). 실측 로그에서 본 `unit=253 (PIDOSETa) / unit=254 (pidomemo`, 둘 다 `msgK=38=0x26=msgK_CTRL`)이 이 경로다 — `screen->m_key` 값이 아니라 별도 록업키이므로, 새 플랫폼에서 두 msgK 그룹을 동일한 방식으로 디코딩하면 안 된다.

4. 화면(맵) 로드/조립 규칙 — CScreen::Parse()

근거: Wizard/Screen.cpp:210~ (실측 로그로 검증됨)

화면 하나가 열릴 때 5단계로 조립됩니다:

1. **LoadForm(mainRc)** — .map 바이너리 파싱, 컨트롤(24종, CfmBase 파생) 실제 생성. `axisform.dll` 의 `CAxisForm::LoadForm` 이 `kind` 값으로 팩토리 분기(`switch(kind){case FM_EDIT: new CfmEdit(...); ...}`)
2. **스크립트 엔진 준비** — `m_vbe->Initialize()`, 아직 VBS/Python 미확정
3. **전역 객체 등록** — `Screen ( CxScreen ) / System ( CxSystem ) / Login / Ledger` 등을 스크립트에 노출 (`AddObject`, 엔진 미확정 이라 버퍼링됨)
4. **폼 전체 순회** (`m_mapH->formN` 개) — 컨트롤별 특수처리 + **FM_OBJECT (임베디드 서브맵)면 재귀적으로 Parse() 재호출** + `FA_FLASH` (실시간 대상) 필드 등록 + 컨트롤별 스크립트 호출
5. **LoadScript()** — 스크립트 실제 로드, VBS/Python 엔진 확정, 3단계에서 버퍼링된 객체 일괄 등록

4.1 서브맵 임베딩 규칙 (FM_OBJECT)

실측 검증 (2026-07-25): 메인화면 1개 열었더니 `Parse` 로그가 4번 찍힘 (`IB120100` +서브맵 `IB12010A / IB12010B / IB120120`).

서브맵은 **별도 창이 아니라, 부모 화면의 좌표계 안에서 스케일링된 부분 영역(unitRc)에 그려짐**:

```
hr = (float)mainRc.Width() / (float)m_mapH->width;
vr = (float)mainRc.Height() / (float)m_mapH->height;
unitRc.left  = int(mainRc.left + ((short)form->m_form->left)*hr);
unitRc.top   = int(mainRc.top  + ((short)form->m_form->top)*vr);
...
screen->Parse(resize, ...); // 재귀 호출, 새 CScreen이지만 같은 창(view) 공유
```

중요: `CScreen` 은 자기 소유의 윈도우(`view`)가 없음 — 항상 `m_client->m_view` (소속 `CClient`의 창)를 공유. 창(윈도우) 1개 : 작업영역(`CClient`) 1개 : 화면(`CScreen`) N개(메인+서브) 관계.

4.2 그리드 특수역할 마커 — vals[2] 의 \$\$ / \$? / \$* (Screen.cpp:372-385, 2026-08-14 확인)

⚠ 주의 — 8.10절의 \$ / ? 마커와 이름만 같을 뿐 완전히 다른 메커니즘이다. 8.10절은 와이어 프로토콜(클라이언트→서버 OOP TR 요청 페이로드) 레벨의 마커였고, 이 절은 맵소스 파싱 시점(화면이 열릴 때 CScreen::Parse() 가 .map 바이너리를 읽는 순간)에만 작동하는 UI 역할 지정 마커다. 소스 위치도 다르다(하나는 IBKSConnector , 하나는 Wizard/Screen.cpp) — 우연히 \$ 라는 같은 특수문자를 재사용했을 뿐 서로 무관하다.

CScreen::Parse() 의 FM_GRID 케이스(352-386행)에서, 그리드 컨트롤의 vals[2] (맵소스 빌드 시 지정하는 부가 문자열 값)을 검사해서 그 그리드가 세 가지 특수 역할 중 하나를 말는지 판별한다:

```
if (form->m_form->vals[2] != NOVALUE)
{
    form->m_form->vals[2] = (DWORD)&m_strR[form->m_form->vals[2]];
    text = (char *)form->m_form->vals[2];
    if (text.Find("$") == 0)
    {
        form->m_form->vals[2] += 2;
        m_notice = form;
    }
    else if (text.Find("$?") == 0)
        m_sales = form;
    else if (text.Find("$*") == 0)
        m_push = form;
}
```

마커	대상 멤버	실제 소비하는 함수	역할
\$\$	m_notice	CScreen::OnNotice() (Screen.cpp:1188)	msgK_TICK (0x90, 1절) 브로드캐스트 알림(체결통보 등, 요청 없이 서버가 미는 데이터)을 표시하는 그리드로 지정
\$?	m_sales	CScreen::UpdateRTM() / ScrollRTM() (Screen.cpp:897, 1132)	RTM 틱 수신 시 새 행을 스크롤 삽입하는 관심종목형 확장 그리드로 지정
\$*	m_push	CScreen::OnPush() (Screen.cpp:719)	푸시 메시지 전용 그리드로 지정

정정(2026-08-14): 이전 대화에서 이 마커를 text.Find("\$") == 0 (홀따옴표 \$ 하나)로 설명한 적이 있는데, 실제 소스는 ** "\$\$" (더블 달러)**다. 단순 \$ 로 시작하고 \$\$ / \$? / \$* 중 아무것도 아닌 vals[2] 값은 이 세 역할 중 어디에도 매칭되지 않고 그냥 지나간다.

5. 컨트롤은 실제 윈도우가 아님 — 가상 컨트롤 구조

근거: dll/form/fmBase.h:124 — class CfmBase : public CCmdTarget (⚠ CWnd 아님)

24종 컨트롤(CfmEdit / CfmGrid / CfmButton /...)은 진짜 Win32 자식 윈도우가 아니라, 부모 창(view) 위에 좌표 기반으로 그려지는 논리적 객체입니다.

메커니즘	방식
그리기	CfmBase::Draw(CDC* dc) — 부모의 DC에 직접 그림 (오너드로우)
클릭 판정	CMouse::WhichForm(CScreen*, CPoint pt) — 좌표로 어느 컨트롤인지 직접 계산 (Windows가 자동 라우팅 안 해줌)
포커스 추적	CCaret (m_key , m_idx) — Win32 SetFocus 가 아니라 직접 변수로 관리

새 플랫폼 설계 시사점: 컨트롤 개수가 많은 화면(그리드 등)에서 네이티브 위젯을 화면 요소 개수만큼 만들면 안 되고, 좌표 기반 커스텀 렌더링 + 수동 히트테스트 방식을 채택한 이유가 성능(리소스 절약)으로 추정됨.

6. 입력 이벤트 → 원시 메시지 후킹 규칙 — 윈도우 서브클래싱

근거: Works.cpp:35,40-41 , Client.cpp:54-56 , Event.cpp:14

```
// 1) 창 생성 시 소유자(CClient*) 저장
SetWindowLong(hwnd, GWL_USERDATA, (long)this);

// 2) 원래 윈도우 프로시저를 CallProc로 교체 (서브클래싱), 원본은 백업
m_callproc = (FARPROC)SetWindowLong(hwnd, GWL_WNDPROC, (LONG)callproc);

// 3) 소멸 시 원본 프로시저로 복원
if (m_callproc) SetWindowLong(hwnd, GWL_WNDPROC, (LONG)m_callproc);
```

Event.cpp 의 CallProc(hwnd, msg, wParam, lParam) 가 그 창의 모든 Win32 메시지를 대신 받아서 GetWindowLong(hwnd, GWL_USERDATA) 로 복원한 CClient* 를 통해 하위 객체로 위임합니다.

CallProc 이 처리하는 메시지 전체 목록 (27종, Event.cpp 실측)

카테고리	메시지	위임 대상
생명주기	WM_DESTROY	client->OnClose()
그리기	WM_PAINT , WM_ERASEBKGD	client->OnDraw()
크기/ 스크롤	WM_SIZE , WM_VSCROLL , WM_HSCROLL , WM_SETCURSOR	
포커스	WM_SETFOCUS , WM_KILLFOCUS	
마우스클릭	WM_LBUTTONDOWN / MBUTTONDOWN / RBUTTONDOWN , *UP , WM_LBUTTONDBLCLK	client->m_mouse->OnDown/OnUp/OnDbClick
마우스이동	WM_MOUSEMOVE , WM_MOUSEWHEEL , WM_MOUSELEAVE	
키보드	WM_CHAR , WM_KEYDOWN	client->m_keyx->OnChar/OnKey

카테고리	메시지	위임 대상
IME	WM_IME_CHAR , WM_IME_COMPOSITION , WM_IME_NOTIFY	한글 조합입력
타이머	WM_TIMER	wParam (타이머ID)로 재분기 — TM_RTM / TM_WAIT / TM_REPBN / TM_REPTR / TM_VB / TM_VBx
커스텀	WM_USER	Wizard 자체 정의 메시지 (미상세조사)
오너드로우	WM_MEASUREITEM , WM_DRAWITEM	커스텀 리스트박스 등

7. 이벤트 → 스크립트 실행 규칙

근거: Wizard/Script.cpp::getProcName , 실측 로그(2026-07-25)

7.1 프로시저 명명 규칙

이벤트	프로시저명 패턴	비고
화면단위 (OnStart/OnClose/OnSend/OnReceive/OnAlert/OnTimer 등)	AX_SUB_On{이벤트}_AX_	심볼 없음
컨트롤단위(OnClick/OnChange/OnDblick 등)	AX_{컨트롤명}_On{이벤트}_AX_	예: AX_BUTTON0_OnClick_AX_
스크립트 내부 임의 서브루틴 호출	이름 그대로 (Screen.Proc("이름", data))	실측: Send_f , OnClear , f_vSendUiData 등 — 표준 이벤트명이 아닌 개발자 정의 함수명

7.2 실행 게이팅 — IsAvailable

```

CString procs = getProcName(evClick, form->GetSymbolName());
if (screen->m_vbe->IsAvailable(procs)) // 스크립트에 이 이름의 함수가 "정의되어 있어야만"
    screen->m_vbe->DoProcedure(procs);

```

빌더의 스크립트 편집창 드롭다운 (OnStart/OnSend/OnReceive/OnAlert/OnService/OnFile/OnSelect/OnTimer/OnTimerX/OnFocus/OnClose 등)에서 실제로 코드를 채운 이벤트만 이 게이트를 통과합니다. 빈 채로 두면 IsAvailable 이 false라 아예 호출 자체가 안 됨(로그도 안 찍힘) — 성능 최적화 + 새 플랫폼에서도 "정의 안 된 이벤트는 스킵"해야 정확히 동일 동작.

7.3 실측 확인된 사실 — 서브맵도 독립적으로 이벤트를 가짐

화면 1개(메인+서브맵 3개) 열었을 때 AX_SUB_OnStart_AX_ 가 2번만 찍힘(4개 화면 중 2개만 실제로 정의) — 각 서브맵은 자기 자신의 독립적인 이벤트 핸들러 세트를 가지며, 메인화면과 무관하게 개별적으로 게이팅됨.

8. 실시간 시세(RTM) 매칭 규칙 — 요약

상세는 @docs/RealtimeCodeIndex_Investigation.md 참고. 핵심만 요약:

- 화면의 "종목코드 필드"는 이름 규칙이 아니라 **FA_FLASH 속성 플래그**로 식별 (FM_EDIT 일 수도 FM_OUT 일 수도 있음 — 조 회용 필드와 실시간 키 필드가 별개인 경우 있음)
- 매칭은 **캐시 없이 매 틱마다 라이브로 필드값을 읽어서(ReadData) 비교**하는 방식 (CScreen::OnAlert)
- 코드값이 바뀌는 경로가 4갈래(직접입력/도미노/TR응답/스크립트SetData)라 개별 후킹이 비현실적 — 원본 구현은 "매 틱 전체 화면 순회 후 비교"라는 O(N) 방식을 씀 (성능 이슈로 별도 개선 작업 진행 중)

새 플랫폼 설계 시사점: 새로 설계한다면 처음부터 "종목코드 → 관심 화면"의 역인덱스를 유지하는 구조로 만드는 게, 지금 겪고 있는 O(N) 문제를 원천적으로 피하는 길.

8.5. 데이터 처리부 상세 — CStream (Stream.cpp)

호출 체인 (전체, 정정판):

소켓 도착

```
→ CWizardCtrl::OnAxis(_axisH*, ...) ① winK로 대상 창 탐색, 서버지시 새창생성/닫기
  → works->OnStream(...) ② CWorks::OnStream, 조각 재조립(statCON)
    → works->OnAxis(...) ③ 가상함수 - CClient::OnAxis는 한 줄 위임(m_stream->OutStream 호출)
      → m_stream->OutStream(...) ④ CStream::OutStream - 실제 분기/파싱 시작
```

①~③은 라우팅/재조립만 하고, ④에서 처음으로 데이터 내용에 따라 분기한다. 반대 방향(클라이언트→서버 송신)은 CGuard::Write / Login / Service 이 담당하고, CClient::OnWrite 에서 보이는 m_stream->InStream(screen) 이 그 카운터파트로 추정됨(필드 변경 시 자동 전송) — Out / In 네이밍은 서버 기준(서버에서 나가는/서버로 들어가는)으로 추정.

정정(2026-07-31): 위 ①(CWizardCtrl::OnAxis)이 실제로는 "첫 분기점"이 아니다 — 더 앞단인 CWizardCtrl::OnRead (소켓에서 막 재조립한 프레임을 받는 지점, 복호화도 여기서 일어남·8.9절)에 msgK 전체를 보는 1차 switch가 있고, 이 표에서 다루는 ①~④ 체인은 그중 TR 요청-응답 계열(AXIS / TAB / HTM / SVC / APC / CTRL / UPF / DNF)만 OnAxis 로 넘겨받은 것이다. 전체 msgK 공간과 1차/2차 분기 구조는 1절 표 참고.

msgK 종류 — OutStream 까지 도달하는 것만 (TR 요청-응답 계열 하위 집합, CStream::OutStream 실측)

전체 msgK 값 목록(SIGN / TICK / ENC 등 OutStream 을 안 타는 것 포함)은 1절 msgK 표 참고 — 이 표는 그중 OutStream 까지 도달하는 것들의 세부 처리만 다룬다.

msgK	값	처리	공통꼬리(커서복원/재조회) 통과 여부
msgK_AXIS	0x20	메인 데이터 — SetDataOOP / SetDataNRM / SetDataNRM2 분기, 끝에 OnReceive 발생	○

msgK	값	처리	공통꼬리(커서복원/재조회) 통과 여부
msgK_HTM	0x21	HTML 뷰로 그대로 전달	X (break 없이 처리 후 흐름 이어짐)
msgK_TAB	0x22	탭구분 데이터 — SetDataTAB / SetDataTAB2	O
msgK_SVC	0x24	서비스콜 — _xscreen->OnService() 우선, 아니면 스크립트 OnService	X (return)
msgK_APC	0x25	승인콜 — 스크립트 OnApprove	X (return)
msgK_CTRL	0x26	FM_CONTROL 필드에 직접 WriteData	X (return)
msgK_UPF / msgK_DNF	0x27/0x28	파일 업/다운로드 — screen->OnFile()	X (return)
msgK_ERR	-	에러 메시지 → SetGuide 로 안내 표시	O
default	-	무시	-

공통꼬리(msgK_AXIS/TAB/ERR만 해당): 커서 위치 복원(statNOC), 추가 안내메시지, 대기상태 해제, statREP 플래그 있으면 RepeatTR (자동 재조회) 트리거.

필드 단위 파싱 알고리즘 — SetDataNRM (Stream.cpp:1474, msgK_AXIS 의 기본 포맷)

- SetDataH()로 레코드 헤더 파싱 (skip=true면 생략, 재귀 호출 시 사용)
- idx가 끝날 때까지 반복:
 - ParseCC()로 현재 바이트가 "제어코드"(FCC/RCC/SCC)인지 확인
 - 제어코드면: 색상/서식 지정으로 추정되는 처리 후 continue (필드 데이터 아님)
 - 아니면: key(필드 순번)를 얻음
 - key를 screen->m_ioR[key]에 대입 → 실제 CfmBase*(폼) 획득
 - 폼 종류(kind)별로 다른 길이/파싱 규칙 적용 (아래 표)

컨트롤 종류별 파싱 규칙:

kind	길이 결정 방식	비고
FM_EDIT / FM_COMBO / FM_OUT	고정길이 — .map 정의의 form->m_form->size	EIO_INPUT (입력전용) 필드는 건너뛴 — 서버 데이터로 안 덮어씀
FM_MEMO / FM_BROWSER	가변길이, 길이 프리픽스 — 앞에 L_FILED 바이트짜리 ASCII 숫자로 뒤따르는 실데이터 길이 표시	FM_BROWSER 는 EIO_OUTPUT 일 때만
FM_BUTTON (라디오)	1바이트 — '0' 이면 미체크, 그 외 체크	EIO_OUTPUT / EIO_INOUT 만
FM_GRID	헤더(옵션, GO_HEADER) + 행수(고정 or GO_FLEX 면 가변길이 프리픽스) + 셀데이터 → SetTable / SetCells 에 위임	그리드 자체가 복합 포맷, 별도 상세조사 필요

kind	길이 결정 방식	비고
FM_TABLE	screen->m_cellR[...] 사전정의 배열의 행별 고정 크기만큼 순회	EIO_NOP 이면 스킵
FM_CONTROL	EIO_INOUT / EIO_OUTPUT 일 때 정의크기(0이면 "남은 전체")	
FM_OBJECT	재귀 호출 — 같은 스트림 안에 서브화면(uob) 데이터가 이어붙어 있음	SetDataNRM(서브screen, ...) 재귀

입력전용 필드 보호 규칙이 여기도 적용됨: RTM(실시간) 경로에서 확인했던 " EIO_INPUT 필드는 서버 데이터로 절대 안 덮어 씌" 규칙이, TR 응답 파싱(SetDataNRM)에서도 동일하게 지켜짐 — 사용자가 입력 중인 필드는 어떤 경로로 와도 보호됨. **새 플랫폼에서 반드시 동일하게 지켜야 할 규칙.**

정확한 판단 기준은 kind 가 아니라 iok (입출력 속성): FM_EDIT / FM_COMBO 도 iok 가 EIO_OUTPUT / EIO_INOUT 이면 FM_OUT 과 동일한 경로로 서버 데이터를 그대로 받는다(2026-07-25 실측 확인, Stream.cpp:1569-1587). 오직 순수 EIO_INPUT 일 때만 스 킵됨. 즉 "EDIT류는 안 받고 OUT류만 받는다"가 아니라 *****iok가 INPUT인 필드만 예외적으로 제외된다*****가 정확한 규칙.

부수 발견 — IsChanged() 호출 여부도 갈림: FM_EDIT / FM_COMBO (비입력전용)는 서버 데이터 반영 후 form->IsChanged() 가 호 출되어 "변경됨" 상태가 되지만(스크립트 onChange 연계 가능), 순수 FM_OUT 은 이 호출이 없음(isSup 플래그로 분기). 출력검 용 입력창과 단순 표시필드를 구분해서 처리하는 것으로 보임 — 새 플랫폼에서 "서버 데이터 반영 시 onChange를 트리거할 지"를 컨트롤 종류(EDIT/COMBO vs OUT)로 판단해야 함.

인밴드 제어코드 (FCC/RCC/SCC) — 2026-07-26 정정: 색상 아니라 "속성 제어"

h/axis.h:280,295,316 — 데이터 스트림 안에 인쇄 불가능한 ASCII 제어문자를 마커로 심어서 일반 필드값과 구분:

마커	값	실제 용도 (CStream::ParseFCC / SetCC , Stream.cpp:2887-2928 실측 확인)
FCC	0x1A	필드/셀 속성을 동적으로 토글하는 제어코드. SetCC() 가 FA_PROTECT (보호/읽기전용), FA_MAND (필수입력), FA_SEND (자동전송), 보이기(SetVisible), 활성화 여부를 서버 응답 도중에 즉석으로 바꿀 수 있음. 색상 지정으로 추정 → 오判定, 색상 아님.
RCC	0x1B	이름으로 지정된 컬럼 (rcc->name , form->GetEnum() 으로 컬럼 인덱스 역조회)에 대해 행 단위 속성 제어 (CScreen::ParseRCC , Screen.cpp:3205)
SCC	0x1C	FCC 와 유사한 구조로 셀 단위 속성 제어(ParseSCC), 상세 미조사

중요 — 새 플랫폼 설계 시사점: 이 마커들은 그림 그리듯 색상을 입히는 게 아니라, **서버가 실시간으로 "이 필드를 지금부터 읽 기전용으로 바꿔라/숨겨라/필수입력으로 만들어라"** 같은 UI 상태를 원격 제어하는 메커니즘입니다. 새 플랫폼에서 그리드/폼 컴포넌트가 이런 동적 속성 변경을 지원하지 않으면, 서버가 의도한 UI 동작(예: 조건 충족 시 필드 잠금)이 재현되지 않습니다.

값이 일반 텍스트/숫자 필드에 나올 수 없는 제어문자 범위(0x1A~0x1C)라 마커로 안전하게 구분 가능 — 새 플랫폼 파서도 이 3바이트 값을 필드 데이터와 구분해서 스킵/해석해야 함.

8.6. 그리드 셀 데이터 파싱 — setCells / SetTable (Stream.cpp)

FM_GRID (GO_TABLE 미설정 시 SetCells , 설정 시 SetTable — 8.5절의 SetDataNRM FM_GRID 분기 참고)의 셀 데이터 포맷.

SetCells (Stream.cpp:2444) — 일반 그리드

```
for row in nRows:
    for col in nCols:
        if cell[col].attr & FA_SKIP: continue           // 스킵 컬럼은 바이트 소비 안 함
        if cell[col].iok != EIO_INOUT/EIO_OUTPUT:
            continue                                     // ★ INPUT 전용 컬럼은 와이어에 아예 안 실림(바이트 자체가 없음)
        formL = cell[col].size                           // 고정길이(컬럼 정의값)
        text += 그값 + '\t'
    text += '\n'
form->WriteAll(text)  // 탭+개행 구분 문자열 통째로 컨트롤에 전달, 파싱은 컨트롤 몫
```

핵심 규칙 — 일반 필드와 다른 점: SetDataNRM 의 EDIT/OUT 필드는 "입력전용이면 읽되 스킵"이었지만, **그리드 컬럼은 입력전용이면 애초에 서버가 그 바이트 자체를 안 보냅니다** (컬럼이 와이어 포맷에서 통째로 빠짐, continue 가 idx 증가 없이 실행됨). 새 플랫폼에서 그리드 파서를 만들 때 이 차이를 놓치면 컬럼이 밀리는 버그가 남.

SetTable (Stream.cpp:2647) — GO_TABLE 방식, 인밴드 제어코드 처리 위치

SetCells 와 기본 구조는 같으나, 셀 하나하나를 읽기 전에 FCC / RCC / SCC 여부를 먼저 검사한다:

```
while (idx + skip < datL)
{
    switch (datB[skip])
    {
        case FCC: ParseFCC(screen, form, ..., col=kk, row=ii); skip += L_FCC; continue;
        case RCC: skip += screen->ParseRCC(form, ..., row=ii); continue;
        case SCC: ParseSCC(screen, form, ..., col=kk, row=ii); skip += L_SCC; continue;
    }
    // 제어코드가 아니면 비로소 실제 셀 값 읽기
    text += CString(&datB[skip], cell->size);
    skip += cell->size;
    break;
}
```

이게 FCC/RCC/SCC가 (col,row) 좌표와 함께 파싱되는 근거 — 그리드/테이블 컨텍스트에서 "몇 번째 행, 몇 번째 열의 속성을 바꿔라"는 지시로 쓰인다는 게 코드로 확정됨. SetCells (GO_TABLE 미설정)에는 이 제어코드 검사 루프가 없음 — GO_TABLE 방식에서만 셀별 동적 속성제어가 지원되는 것으로 보임(추가확인 필요).

사용자 정의 컬럼 변형 — SetCells2 / SetTable2

컬럼을 사용자가 재배열/숨김 설정한 경우(screen->m_ocols 에 컬럼 매핑 정보 존재 시) 쓰이는 변형. CtranItem::m_col < 0 이면 "서버는 이 컬럼 데이터를 여전히 보내지만 사용자 설정상 숨겨진 컬럼"이라는 뜻으로, **바이트는 소비하되 (idx += citem->m_size) 텍스트에는 안 넣음** — SetCells (입력전용 컬럼은 바이트 자체가 없음)와는 다른 종류의 "스킵"이라 새 플랫폼에서 혼동 주의.

8.7. 레코드 헤더(setDataH) — 원장(Ledger) 블록의 실제 위치 (2026-07-26 실측 확인)

핵심 발견: 화면의 맵 타입(m_mapH->typeH)이 TH_LEDGER 인 경우, 페이로드 구조는 다음과 같습니다:

[axisH 헤더 24바이트] + [원장(Ledger) 블록, screen->m_ledgerL 바이트] + [일반 TR 필드 데이터 (8.5절 setDataNRM)]

CStream::setDataH (Stream.cpp:2410, setDataNRM 이 매 레코드 시작 시 제일 먼저 호출)가 이 분기를 담당:

```
bool CStream::setDataH(CScreen* screen, char* datB, int& datH)
{
    datH = 0;
    switch (screen->m_mapH->typeH)
    {
        case TH_LEDGER:
            datH += screen->m_ledgerL;          // 원장 블록 크기만큼 idx를 밀어냄
            screen->SetLedger(datB);           // 원장 블록을 외부 DLL(dll/login)에 그대로 전달
            skip = atoi(m_guard->GetLedger(screen->m_ledger, getOK)); // 처리결과 재확인
            if (skip == 0) return false;       // 실패 시 전체 파싱 중단
            if (skip < 0) datH += -skip;       // 추가 스킵량
            break;
        // TH_KOSCOM/TH_SCUP/TH_4702는 별도 처리 필요해보이나 현재 분기는 비어있음(추가조사 필요)
    }
    return true;
}
```

원장(Ledger) 자체는 Wizard가 파싱하지 않음 — 별도 DLL(dll/login/)의 블랙박스: screen->SetLedger(datB) 는 원장 블록의 바이트를 그대로 외부 DLL(axSetLedger / axLedger 등 함수포인터, dll/login/ledger.h · ledgerx.cpp)에 넘길 뿐, Wizard 자체는 그 내부구조를 해석하지 않습니다. 이후 화면단 필요 시 GetLedger(ledger, pos, length) / GetLedger(ledger, id) 로 위치/ID 기반 조회만 함.

원장 헤더 구조체 _ledgerH — 금융권 표준 "공통전문헤더" 포맷 (h/ledger.h)

기본은행/증권 호스트 인터페이스의 전형적인 공통 헤더 포맷(전문길이/서비스코드/사용자ID/지점번호/단말번호/기능키/연속키/오류코드/오류메시지 등). 주요 필드:

필드	오프셋	크기	의미
size	0	6	전문 전체 길이
type	6	1	전문구분(B:계정계, I:정보계, F:파일송수신, N:Notice)
svcd	12	8	서비스코드
usid	23	16	사용자ID
pbid	53	16	트랜잭션추적ID
term	99	8	단말번호
ecod	117	4	메시지코드(오류코드, 0000~0999 정상/1000~9999 비정상)

필드	오프셋	크기	의미
fkey	123	4	기능키(5:확인 7:연속 C:전송)
next	137	1	연속여부(Y:연속있음)
nkey	138	18	연속처리 키값
mkty / odrf	172/173	1/1	시장구분(1:현물 2:선옵 3:기타) / 주문구분(1:주문 2:조회 3:이체)
emsg	241	99	오류메시지(코드4+메시지94)

⚠ **코드베이스 내 구조체 정의 불일치 발견:** h/ledger.h (공용 헤더, **340바이트**, epwd 필드 없음)와 Wizard/Client.cpp:4243 (함수 내부 로컬 재정의, **384바이트**, epwd[44] —sha256 암호화 계좌비밀번호 필드 추가로 있음)가 서로 다릅니다. epwd 가 최근에 추가된 필드인데 공용 헤더(h/ledger.h)에는 아직 반영이 안 된 것으로 보임 — 실제 원장 데이터가 340/384바이트 중 어느 쪽인지는 screen->m_ledgerL 을 로그로 실측해서 확인 필요(로그 추가함, 아래 참고).

실측 결과 (2026-07-26 확인 완료): IB12010 (계좌잔고 성격 화면) 조회 시 [SetDataH] typeH=6 ledgerL=384 로그로 확인됨 — 실제 운영 데이터는 **384바이트(epwd 포함) 버전이 맞다.** 즉 h/ledger.h (340바이트, 공용헤더)가 구버전이고, Client.cpp 로컬 정의(384바이트)가 현재 실제 프로토콜과 일치. **새 플랫폼에서는 384바이트 버전(epwd 포함)을 기준으로 구현해야 함.** typeH=6 이 TH_LEDGER 상수값으로 확인됨(추가 매핑 정보는 axisfm.h/axisfire.h류에서 TH_LEDGER define 값 대조 필요).

8.8. 송신측 TCP write 프레이밍 — 한 번의 write에 여러 _axisH 메시지가 묶일 수 있음 (2026-07-30 실측 확인)

CGuard::Write(char* pBytes, int nBytes, bool trace) 가 실제 소켓 송신 직전의 **유일한 합류 지점**임을 확인함(RouteTR (일반 TR)과 CGuard::UploadFile (파일 업로드) 둘 다 최종적으로 이 함수 하나를 거침). 여기에 로그를 달아보니, **한 번의 Write() 호출(=한 번의 TCP 송신)이 서로 다른 화면(unit) 앞으로 가는 _axisH 프레임을 여러 개 이어붙여서 보낼 수 있다**는 게 실측으로 확인됨.

원인: CStream::MakeStream(bool byKey) (단일 화면용 MakeStream(CScreen*, CString) 과는 다른 오버로드)는 m_client 에 속한 모든 화면을 순회하며(skip / MM_MENU / isUob() 제외) 화면마다 MakeStream(screen) 을 호출해 **같은 송신버퍼(m_sndB)에 계속 이어붙인다.** m_sndL (누적 오프셋)은 이 배치 시작 시점에 딱 한 번만 0으로 리셋되고, 그 뒤 화면 개수만큼 _axisH +데이터 프레임이 순서대로 쌓인 뒤, **딱 한 번의 RouteTR / Write() 로 전부 한꺼번에 전송된다.** 이 배치 경로는 스크립트의 Screen.Send(targetALL) (CxScreen::_Send 의 targetALL case → 인자 없는 InStream() 호출)로 진입한다.

실측 예시(2026-07-30): 화면 안에 IB282300 (unit=0)/ IB282310 (unit=1) 두 화면이 동시에 조회를 시도한 경우:

```
[0-MakeStream-send] ----- map=IB282300 tr=poopoop ----- winK=47 unit=0
[0-MakeStream-send] ----- map=IB282310 tr=poopoop ----- winK=47 unit=1
[0-Write-send] #0 winK=47 unit=0 msgK=32 trxC=poopoop datL=... preview=[...]
[0-Write-send] #1 winK=47 unit=1 msgK=32 trxC=poopoop datL=... preview=[...]
CGuard::Write(2) len=315 ...      <- 두 프레임 합쳐서 한 번에 315바이트 전송
```

AxisChaser는 이 315바이트짜리 TCP 페이로드를 각 프레임의 datL (헤더의 5자리 ASCII 길이 필드) 경계로 스스로 파싱해서 [Send Data 87 Bytes][Unit 0] / [Send Data 228 Bytes][Unit 1] 두 개의 별도 항목으로 나눠 보여준다 — **Wizard 쪽에서 보면 한 번의 소켓 write이지만, 논리적으로는 독립된 여러 메시지가 프레임 단위로 이어붙여진 것이다.**

새 플랫폼 설계 시사점: 소켓 수신 파서를 만들 때 "한 번의 recv/read가 정확히 하나의 논리 메시지"라고 가정하면 안 된다. TCP 스트림 특성상 여러 프레임이 한 번의 시스템 콜로 도착하거나(반대로 하나의 프레임이 여러 번에 걸쳐 도착하거나, 4절의 statCON 재조립 규칙 참고) 할 수 있으므로, 항상 `_axisH` 헤더의 `datL` 필드로 프레임 경계를 직접 계산하며 루프를 도는 파서가 필요하다 — 이번 확인으로 송신측도 동일한 원칙이 적용됨을 알았으니, 신규 구현체는 송수신 양쪽 모두 "고정 24바이트 헤더 + `datL` 만큼의 페이로드"를 한 단위로 보고 반복 파싱하는 구조로 설계해야 한다.

8.9. 페이로드 암호화/복호화 — CGuard::Xecure (2026-07-31 실측 확인)

실측 캡처 (IB140300, SRGSQ145 조회 — 결산분배금 내역):

[0-MakeStream-send] map=IB140300 tr=piboPBxQ winK=39 unit=0	
[0-GetDataNRM-field] name=zPwd kind=7 value=[***]	(평균 조립, <code>L_axisH</code> 이후 페이로드)
[Xecure] helper=ENC nBytesIn=434 nBytesOut=496 retv=1	<- 암호화: 434B -> 496B
[0-Write-send] ... datL=496 (암호문 그대로 전송)	
CGuard::Write(2) len=520 hdr0=0x20 hdr3=0x27	<- $520 = L_axisH(24) + 496$, 정확히 일치
...	
[Xecure] helper=DEC nBytesIn=912 nBytesOut=860 retv=1	<- 응답 복호화: 912B -> 860B
[1-OnAxis-raw] nBytes=860	<- 이후 파이프라인은 평균 기준으로 정상 진행

확정된 규칙

- 암호화 대상은 `_axisH` 헤더(24바이트) 이후 페이로드뿐 — 헤더 자체는 항상 평문이다.**
CStream::MakeStream / Guard.cpp 의 모든 Xecure(DI_ENC, ...) 호출은 `&sendB[L_axisH]` / `&m_sndB[m_sndL]` (헤더 다음 위치)부터 시작하는 버퍼를 넘긴다. `winK / unit / trxC / datL` 같은 라우팅 정보를 서버가 복호화 없이도 읽을 수 있어야 하므로 당연한 설계지만, 이번에 바이트 길이 계산으로 실측 확정됨: `CGuard::Write(2) len=520` = 헤더 24B + 암호문 496B, 오차 없음.
- 암호화 여부는 헤더의 `stat` 필드 `statENC` 비트로 표시된다.** 송신측이 Xecure(DI_ENC, ...) 성공 시 `axisH->stat |= statENC` 를 세팅하고, 수신측(`CWizardCtrl::OnRead` , `WizardCtrl.cpp:712`)은 `[1-OnAxis-raw]` 로 넘기기 전에 `axisH->stat & statENC` 를 검사해 켜져 있으면 Xecure(DI_DEC, ...) 부터 통과시킨다. 복호화 실패 시 그 프레임은 조용히 버려진다(`continue` , [Xecure] decrypt FAILED 로그 참고, docs/DebugLogGuide.md 12절).
- 암호화는 전체 TR이 아니라 맵(화면) 단위로 켜고 끈다.** CStream::MakeStream 의 게이트 조건은 `!(m_guard->m_term & flagENX) && screen->m_mapH->options & OP_ENC` — 즉 (a) 이 맵의 빌드 옵션에 `OP_ENC` 가 설정돼 있어야 하고, (b) 로그인 시 받은 단말 권한 플래그(`m_term`)에 암호화 예외(`flagENX`) 비트가 없어야 한다. 두 조건 다 충족해야만 암호화 경로를 탄다 — 모든 TR이 무조건 암호화되는 게 아니다.
- datL (헤더의 5자리 ASCII 길이 필드)는 암호화가 걸리면 원문이 아니라 암호문 길이를 담는다.** 위 캡처에서 원문 434B가 아니라 암호문 496B가 `datL / Write` 길이에 그대로 반영됨 — 새 플랫폼 파서는 "헤더 다음에 `datL` 만큼 읽고, `statENC` 가 켜져 있으면 그 블록 전체를 복호화한 뒤에야 실제 필드 파싱(`SetDataNRM` 등)을 시작"하는 순서를 지켜야 한다.
- 암호화/복호화 알고리즘 자체는 이번 조사로 확인 불가 — AxisXecure.XecureCtrl.IBK2019 라는 서드파티 ActiveX 컨트롤 (COM InvokeHelper 호출) 뒤에 완전히 캡슐화돼 있다.** `nBytesIn` → `nBytesOut` 증가폭(이번 예: 요청 434→496, +62B / 응답 912→860, -52B)은 매번 다를 수 있어 보이며(요청과 응답은 별개의 메시지라 증감폭이 대칭일 필요는 없음), 패딩 +IV+MAC 등 정확한 구성은 블랙박스 안에 있어 바이트 길이 관찰만으로는 알고리즘을 특정할 수 없다. 새 플랫폼에서 동일 프로토콜을 구현하려면 이 컨트롤을 그대로 재사용하거나(COM 상호운용), 벤더/보안팀으로부터 실제 알고리즘·키 교환 방식을 별도로 전달받아야 한다 — 리버스엔지니어링 대상이 아니라 공식 스펙을 받아야 하는 영역.
- 서버는 클라이언트가 보낸 요청의 `statENC` 비트를 보고 응답의 암호화 여부를 그대로 맞춘다.** 같은 화면/TR(IB140300, SRGSQ145)을 암호화 켜 상태와 끈 상태로 번갈아 조회해서 확인(2026-07-31, docs/DebugLogGuide.md 12절의

NOENC.TXT 개발용 스위치로 재현) — 요청에 statENC 를 안 실행하면(암호화 생략) 응답도 stat=0 (평문)으로 오고, 요청에 statENC 를 실행하면 응답도 stat=2 로 암호화돼서 온다. 즉 암호화 여부는 세션/계정 단위 서버 정책이 아니라 매 요청마다 클라이언트가 선언한 값에 서버가 그대로 반응하는 요청-단위(per-request) 협상이다 — 새 플랫폼도 "요청 시점에 암호화 여부를 결정해 stat 비트로 선언하면, 그 요청에 대한 응답은 동일한 방식으로 온다"고 가정하고 구현하면 된다.

개발용 진단 스위치 — NOENC.TXT

호스트 exe(Axis.exe) 폴더에 NOENC.TXT 빈 파일을 두면(CGuard::IsNoEncMode() , 앱 재시작 불필요, 파일 유무를 매번 재확인) 위 규칙 6에 따라 요청·응답이 전부 평문으로 오가게 되어 로그에서 실제 필드 데이터를 바로 읽을 수 있다 — 이번 절의 확인들도 이 스위치로 재현한 것. 상세는 docs/DebugLogGuide.md 12절.

로그 대응표

docs/DebugLogGuide.md 12절에 상세 — [Xecure] 키워드로 DebugView 필터링하면 이 절의 모든 호출이 잡힌다.

8.10. IBKSCConnector OOP 요청 페이로드 문법 — pooppoop / GOOPHOOP 계열 (2026-08-14 확인)

배경 — 이 절이 다루는 범위

8.9절까지는 trxC=piboPBxQ (US_ENC , 일반 AXIS 포맷)류를 다뤘다. 이 절은 trxC 가 pooppoop / GOOPHOOP 등인 US_OOP 포맷 TR의 요청 페이로드를 클라이언트(IBKSCConnector OCX)가 어떻게 조립하는지를 다룬다. MigrationSpec 9절에 오래 남아있던 SetDataOOP (서버 응답을 Wizard가 파싱하는 쪽, Stream.cpp)와는 반대 방향(클라이언트→서버 요청 조립)이며 소스 위치도 다르다 — Wizard/ 안이 아니라 HTS_OpenAPI(운영)/IBKSCConnector/IBKSCConnectorCtl.cpp 에 있다. 즉 OOP 프로토콜의 "요청 조립" 절반은 이번에 확정됐고, "응답 파싱"(SetDataOOP) 절반은 여전히 미확인이다(9절 참고).

⚠ 주의 — 4.2절의 \$\$ / \$? / \$* (그리드 특수역할 마커)와 이름만 같을 뿐 무관한 별개 메커니즘이다. 저건 맵소스 파싱 시점 (CScreen::Parse())의 UI 역할 지정이고, 이 절은 와이어 프로토콜(요청 페이로드) 레벨의 마커다.

중요 — 이 문법은 trxC 자체와 무관하게, _axisH 헤더(24바이트) 다음에 오는 페이로드 안에서 작동하는 문법이다. 즉 1절의 _axisH 구조(헤더 24바이트+ datL 만큼의 페이로드), 8.9절의 암호화 규칙(헤더 이후 페이로드만 암호화)은 OOP 포맷에도 동일하게 적용되고, 이 절은 그 페이로드 내부의 문법만 다룬다.

근거: HTS_OpenAPI(운영)/IBKSCConnector/IBKSCConnectorCtl.cpp 의 S_TR1002 / S_TR1003 / S_TR3002 / S_TR3003 (2341~2505행)과 IBKSCConnector_test/IBKSCConnectorCtl.cpp 의 S_TR1007 (2527~2662행, 원본 CP949 주식 iconv로 확인) — 전부 실제 소스 코드 직접 확인(로그/추정 아님).

1) 단일값(비그리드) 조회 — 특수문자 없음

S_TR1002 (주식 조회), S_TR3002 (선물옵션 조회):

```
sdat.Format("1301\x7f%$t%$t", code, data);  
return SendTR("pooppoop", key, US_OOP, (LPCSTR)sdat, sdat.GetLength(), &IBKSCConnectorCtrl::C_TR1002);
```

문법: <필드코드>\x7f<값>\t<필드코드2>\t<필드코드3>...\t

- 1301 = "종목코드" 필드코드. 여기에 값을 채우고 싶으면 \x7f (0x7f, DEL 문자)로 필드코드와 값을 붙인다:
1301\x7f005930 .
- 값 없이 "이 필드코드를 조회해달라"만 요청할 때는 필드코드만 탭(\t , 0x09)으로 나열한다.
- data (함수 인자명은 파일마다 data / symb 로 다르게 불리지만 실체는 같음, Python dynamicCall 에서는 TR1002(int, QString, QString) 의 세 번째 인자)는 **조회하고 싶은 필드코드들을 탭으로 이어붙인 문자열**이다. "심볼 하나"가 아니라 "필드코드 목록"이다.
- 맨 끝에 항상 trailing 탭이 하나 더 붙는다(Format(...%s\t) 의 마지막 \t).

실측 예시(사용자가 앞서 캡처한 로그)와 대조:

```
preview=[1301024110      1034      1022      1023      1024      1033      1027      1025      1026      1029      1030
```

- 1301024110 = 1301 + \x7f (비인쇄 문자라 화면엔 안 보임) + 024110 (종목코드) — \x7f 가 프린트 안 되는 문자라 로그 미리보기에서는 필드코드와 종목코드가 그냥 붙어 보인 것.
- 그 뒤 1034\t1022\t1023\t1024... 는 조회를 요청한 필드코드 목록 — 실제로 이 18개 목록 (1034,1022,1023,1024,1033,1027,1025,1026,1029,1030,1031,1021,1310,1311,1312,1313,1316,1318)은 s_TR1007 이 내부적으로 쓰는 snapFields[] 배열과 정확히 동일하다(아래 3절 참고) — "스냅샷 18필드"라는 이름으로 여러 TR 호출부에서 재사용되는 공통 필드 세트로 보인다.

사용자 질문에 대한 답: " 1301 0x7f 실제종목코드 0x09 심볼 0x09 — 종목코드는 1301 값이고 심볼에 해당하는 데이터 조회시 사용" — **골격은 정확히 맞다.** 다만 "심볼"이라는 이름 때문에 오해할 수 있는데, 실제로는 단일 심볼이 아니라 **조회하고 싶은 필드코드들을 탭으로 이어붙인 문자열 전체**가 그 자리에 들어간다(위 로그 예시처럼 18개가 한 번에 들어갈 수 있음). 마지막 탭은 종결자다.

2) 그리드(다중 레코드) 조회 — \$ 마커

s_TR1003 (주식 그리드 조회, 예: 체결/호가 등 여러 행이 필요한 조회), s_TR3003 (선물옵션 버전):

```
sin.Format("1301\x7f%s\t10302\x7fd\t$10310\x7f", code, type); // 3003은 "30301.../$33310" 또는 "40301.../$43310"
sout = columns; sout.TrimRight(); sout.Replace('\t', '\n'); sout += "\n\t";
```

```
vector<char> buff(ilen+glen+olen);
memcpy(&buff[0], sin, ilen);
memcpy(&buff[ilen], &gin, glen); // grid_i 이진 구조체
memcpy(&buff[ilen+glen], sout, olen);
```

```
return SendTR("poopoop", key, US_00P, buff.data(), buff.size(), &CIBKSCconnectorCtrl::C_TR1003);
```

문법: <일반필드=값...>\t \$<그리드시작필드코드>\x7f [grid_i 이진구조체] [컬럼목록, \n구분]\n\t

- \$10310 (3003은 \$33310 / \$43310) — **필드코드 앞에 \$ 가 붙으면 "여기부터는 단일 값이 아니라 그리드(표) 형태로 응답을 만들어달라"**는 마커다. 1)의 문법과 필드코드/값 부분은 동일하되, 그리드 시작 지점에만 \$ 가 붙는다.
- \$필드코드\x7f 바로 뒤에는 텍스트가 아니라 이진 구조체 grid_i (grid_i.h)가 그대로 붙는다 — 정렬/페이징 요청 정보:

```

class grid_i {
    char vrow[2];    // 보이는 행수
    char nrow[4];    // 요청 행수
    char vflg[1];    // 뷰 플래그
    char gdir[1];    // 정렬방향(그리드)
    char sdir[1];    // 정렬방향(부가)
    char scol[16];   // 정렬 기준 컬럼명
    char ikey[1];    // 입력 키(연속조회 시 setIKEY(2))
    char page[4];    // 페이지 번호
    char save[80];   // 연속조회 이어보기 토큰 - 응답 grid_o.save를 그대로 되돌려보냄
};

```

- grid_i 바이너리 다음에 원하는 컬럼(필드) 목록이 텍스트로 이어진다 — 호출자가 넘긴 columns (탭구분)를 개행(\n)구분으로 바꾸고 끝에 \n\t 를 붙인 형태.
- 연속조회(페이징) 메커니즘: 응답에는 grid_o (같은 헤더, IsNext() / GetNKey() 제공)가 돌아온다. 다음 페이지가 필요하면 s_TR1003 의 nkey 인자에 직전 응답의 grid_o.save (80바이트)를 그대로 넣어서 재호출 → gin.setGRID0(gout) 으로 grid_i.save 에 그대로 반영됨. 즉 ** save 80바이트가 서버-클라이언트 간 "이어보기 토큰"***이고, 새 플랫폼도 이 필드를 불투명한 opaque 토큰으로 왕복시켜주기만 하면 됨(내용 해석 불필요).

3) 캔들/시계열 조회 — ? 마커

s_TR1007 (차트/캔들 조회, trxC=GOOPHOOP , IBKSConnector_test/IBKSConnectorCtl.cpp:2527):

```

sendS += "1301"; sendS += (char)0x7f; sendS += code; sendS += "\t";
for (18개 snapFields) sendS += 필드코드 + "\t";    // 종목이면 그대로, 업종(GU_INDEX)이면 "2" 접두(21301, 21034...)

sendS += "1777"; sendS += (char)0x7f; sendS += mkgubn; sendS += "\t";

sendS += (dunit==2) ? "?25500" : "?5500";          // ★ 캔들 마커 - $가 아니라 ?
sendS += (char)0x7f;
// _dataH(136바이트 이진 헤더): count[6] + dummy[6] + dkind + dkey + pday[8] + dunit + dindex + lgap[4] + ltic[4]
//                               + option1 + option2 + rcode[16] + ikey + xpos + page[4] + save[80]

for (캔들필드 10개) sendS += 필드코드 + "\n";        // 시가/고가/저가/종가/거래량/거래대금/권리락/수정비율 등
sendS += "\t";

return SendTR("GOOPHOOP", key, US_OOP, sendS.data(), sendS.size(), &CIBKSConnectorCtrl::C_TR1007);

```

문법: 2)와 빠대는 동일(특수문자+필드코드\x7f → 이진헤더 → 필드목록\n구분\n\t)이지만 **마커 문자가 \$가 아니라 ?**다 (5500 / 25500 — 종목은 접두어 없이, 업종은 2 접두). 이진 헤더도 grid_i 가 아니라 TR1007 전용 136바이트 _dataH (요청 건수/기준일/일봉·주봉·월봉 구분 등)로 다르다.

주의 — 업종(GU_INDEX) vs 종목(GU_CODE) 전체 접두 규칙: dunit==2 (업종)이면 코드값 자체는 그대로(3자리, 00 접두 없음)이지만 필드코드/캔들마커 전부에 2가 접두된다(1301 → 21301 , 5500 → 25500 , 캔들필드 5302 → 25302 등).

docs/RealtimeCodeIndex_Investigation.md 류 조사에서 마주칠 수 있는 2로 시작하는 필드코드들이 이 규칙과 관련 있을 가능성이 있다 — 교차 확인 가치 있음.

4) 공통 요약 — 세 문법을 관통하는 규칙

[일반 필드코드\x7f값 / 필드코드만 ...]\t [마커?][그리드또는캔들시작 필드코드]\x7f [이진 헤더구조체] [원하는필드/컬럼목록

구분	마커	이진 헤더	헤더 크기	뒤따르는 목록
단일값 (TR1002/3002)	없음	없음	-	(없음, 앞부분 필드코드 나열이 곧 요청 전부)
그리드 (TR1003/3003)	\$	grid_i	sizeof(grid_i)	컬럼 목록
캔들/시계열 (TR1007)	?	_dataH	136바이트	캔들 필드 목록(10개)

공통 구분자:

- \x7f (0x7f, DEL) — 필드코드와 그 값을 붙일 때
- \t (0x09) — 필드/파라미터 구분
- \n (0x0a) — 그리드/캔들 문법에서만, 목록 항목 구분
- 페이로드 끝은 항상 \t 로 종결

5) 확인 범위와 다음 단계

확인 완료: TR1002, TR1003, TR3002, TR3003, TR1007 — 5개 TR의 요청 조립 문법.

미확인 (다음 관찰 대상):

- \$ / ? 외에 다른 마커 문자가 더 존재하는지 (예: 파일업로드/원장류 등 다른 US_OOP TR)
- 이 문법을 쓰는 다른 그리드성 OOP TR이 더 있는지 — IBKSConnectorCt1.cpp 에서 us_oop 로 검색하면 후보를 추가로 찾을 수 있음
- Wizard(서버) 쪽에서 이 요청을 받아 실제로 어떻게 파싱/응답을 만드는지(setData00P , 9절의 기존 미확인 항목과 동일 — 이 절이 다룬 건 클라이언트 쪽 절반뿐)

진행 방법(사용자 제안, 2026-08-14): AxisChaser로 새로운 그리드성 TR을 캡처할 때마다, "같은 TR의 비그리드 버전(있다면)"이나 "필드코드만 나열한 단순 요청"과 바이트 단위로 대조해서 어느 필드코드 앞에 특수문자가 붙었는지만 찾으면 위 4)의 문법이 그대로 적용될 가능성이 높다. 새로 확인되는 대로 이 절에 TR을 추가할 것.

8.11. 그리드 실시간 갱신 3종 마커 — \$? / \$\$ / \$* (2026-08-17 확인)

배경: CScreen::Parse() (4절)의 FM_GRID 처리 중 form->m_form->vals[2] (빌더 UI의 "Variant" 속성)에 특수 문자열이 들어있으면 그 그리드를 화면당 딱 1개씩 m_notice / m_sales / m_push 중 하나로 등록해두는 코드가 있었음(문서에 등장은 했으나 실제 사용처는 이번에 처음 추적). 이 3개는 서로 완전히 다른 실시간 갱신 메커니즘이며, 사용자가 실제 HTS 화면(체결 그리드, 시가총액순위 그리드)과 대조하며 검증함.

마커	저장 필드	데이터 출처	갱신 방식	용도(추정)
\$?	m_sales	RTM(_rtmH , 6절 요약) 스크롤버퍼	무조건 CfmGrid::InsertRow — 기존 행 매칭 없음	체결 틱커(스크린샷으로 실측 확인)

마커	저장 필드	데이터 출처	갱신 방식	용도(추정)
\$\$	m_notice	RTM(_rtmH)	키 매칭 → 있으면 셀 갱신 or 상태플래그 꺼지면 행삭제, 없으면 행삽입 (upsert)	미체결/주문 리스트류 (추정, 실측화면 미확인)
\$*	m_push	별도 채널(_anmH , 아래 8.11.3)	고정크기 원형버퍼, 커서(m_row) 위치에 덮어쓰기	대량 배치푸시/뉴스크롤류 (추정, 실측화면 미확인)

8.11.1. \$? — CScreen::ScrollRTM (순수 삽입, 매칭 없음)

RTM 파이프라인(1절/6절)에서 종목별로 스크롤형(stat_SCR , 아래 참고) 데이터를 버퍼링했다가 한꺼번에 플러시하는 구조:

```
CGuard::OnAlert(code, pBytes, nBytes, stat)  [Guard.cpp:1030]
    rtmK_INFO 패킷: 종목의 필드번호 순서표(rtmk, CStringArray)를 캐시
    rtmK_DATA 패킷: 값 파싱, 종목코드별 CObArray*(scroll맵, obs)에 CdataSet 버퍼링(InsertAt(0,...))
    → 같은 종목 "일반" 시세갱신이 오거나, 이번 소켓패킷 처리가 끝나는 시점에
        DoRTM(code, stat_SCR, rts, obs, updates) 로 일괄 플러시
        ↓
    CScreen::UpdateRTM (Screen.cpp:897)
        if (m_sales && (stat & alert_SCR)) ScrollRTM(obs);
        ↓
    CScreen::ScrollRTM(CObArray* obs)  [Screen.cpp:1132]
        obs 안 CdataSet들을 컬럼이름으로 조회해 탭구분 멀티라인 문자열로 조립
        (form->m_form->attr2 & GO_TOP) ? InsertRows(0,...) : InsertRows(-1,...)
        ↓
    CfmGrid::InsertRows()  [dll/form/fmGrid.cpp:5016, axisform.dll]
        줄바꿈으로 쪼개 insertRow()로 실제 삽입, 기존 행은 밀려남
```

stat_SCR = 0x10 // scroll data (grid, graph) (h/axisanm.h:59) — 와이어 RTM 헤더(rtmH->stat)에 실제로 실리는 비트,
서버가 "이건 스크롤형 데이터"라고 명시적으로 표시함. **행 매칭 로직이 전혀 없음** — 종목코드나 특정 row를 찾는 코드가
ScrollRTM 안에 없음(실측 확인).

8.11.2. \$\$ — CScreen::OnNotice 후반부 (키매칭 upsert/삭제)

OnNotice(CdataSet& major, CdataSet& minor, CdataSet& fms, CString notices) (Screen.cpp:1188)의 후반부(1288~1397):

```

// 1. minor(키 데이터셋)로 기존 행 중 일치하는 행 탐색
for (kk=0; kk<nRows; kk++)
    for (idx=0; idx<nCols; idx++)
        if (minor.Lookup(form->GetName(idx), string) && text.Compare(string)==0) match=true;

// 2. vals[2]에 남은 숫자("$201" -> Parse())가 "$$" 2글자 skip -> "201")가
//     "삭제여부" 판단용 필드번호
name = atoi((char*)form->m_form->vals[2]);

if (match && fms.Lookup(name,text) && !atoi(text))
    ((CfmGrid*)form)->RemoveRow(kk); // 상태플래그 꺼짐 -> 행 삭제
else if (!match && (!fms.Lookup(name,text) || atoi(text)))
    ((CfmGrid*)form)->InsertRow(GO_TOP ? 0 : -1); // 새 키 -> 행 삽입

if (match)
    for (idx=0; idx<nCols; idx++)
        form->WriteData(text, true, idx, kk); // 그 행 셀들 갱신

```

major 는 화면 레벨 컨텍스트 매칭(같은 함수 앞부분, 1203~1286행), minor 는 그리드 행 단위 키 매칭 — 2단계 스코프 구조.
 실측 화면(어떤 맵이 \$\$ 를 쓰는지)은 아직 미확인 — 동작 특성(키 매칭+상태기반 삭제)상 미체결/주문상태 리스트류로 추정.

8.11.3. \$* — _anmH 라는 완전히 별도의 소켓 채널 (신규 발견)

핵심 발견: \$* 는 RTM(_rtmH)도 TR(_axisH)도 아닌 제3의 독립 프로토콜을 씀. 소켓 OCX가 쓰는 이벤트 자체가 분리되어 있음:

```

// WizardCtrl.cpp:203, CWizardCtrl::OnFireEvent
case FEV_ANM: OnAlert(pBytes, nBytes); break; // RTM ( _rtmH )
case FEV_PUSH: OnPush(pBytes, nBytes); break; // ← $* 전용 채널
case FEV_AXIS: OnRead(pBytes, nBytes); break; // TR ( _axisH )

```

수신 파싱 (CWizardCtrl::OnPush , WizardCtrl.cpp:663) — 자체 헤더 _anmH 사용:

```

struct _anmH* anmH = (struct _anmH*)pBytes;
switch (anmH->anmK) {
case anmK_ALIVE: break; // 하트비트
case anmK_PUSH: m_guard->OnPush(CString(&pBytes[L_anmH], anmL)); break;
}

```

구독 요청(클라이언트→서버) — CGuard::SetPush(bool push) (Guard.cpp:715). \$* 그리드가 있는 화면이 열리면
 (CScreen::Parse() , Screen.cpp:511) push=true 로 호출됨:

```
// CScreen::isPush(pushN) - 화면의 첫 FM_EDIT 필드값을 "토픽 이름"으로 사용
anmH->anmK = anmK_PUSH;
anmH->anmF = push ? 1 : 0; // 구독/해지 플래그
if (!pushN.IsEmpty()) {
    // USRDIR/{pushN} 로컬 파일을 읽어 페이로드에 실어 같이 전송 (동기화용으로 추정)
    fileH.Open(path, ...); fileH.Read(&datB[L_anmH], datL);
}
m_sock->InvokeHelper(DI_DWRITE, ...); // ★ 일반 DI_WRITE(axisH 경로)가 아닌 별도 디스패치
```

수신 후 처리 — CGuard::OnPush(CString pushes) (Guard.cpp:763): 열려있는 모든 CClient (S_LOAD 상태)에 client->OnPush(pushes) 로 브로드캐스트 → CClient::OnPush (Client.cpp:1502) → screen->OnPush(pushes) . 처리 후 SetPush(false) 로 즉시 자동 구독해지 — 지속 스트리밍이 아니라 1회성 배치 수신+해지 패턴.

그리드 반영 — CScreen::OnPush(CString pushes) (Screen.cpp:719): \$? (InsertRow로 밀어내기)와 달리 고정 크기 원형버퍼:

```
form->WriteData(cmps, true, idx, m_row); // 커서(m_row) 위치에 씀
if (++m_row >= rowN) m_row = 0; // 끝까지 가면 처음으로 순환
// 바로 다음 칸은 미리 공백으로 지워둠 (다음 삽입 예정 위치 표시)
```

\x1b (ESC) 이스케이프로 라인별 전경/배경 RGB 직접 지정 가능(text.Mid(1,3) / Mid(4,3) = 3자리 RGB 코드) — 뉴스크롤/공지사항처럼 서버가 색상 포함된 대량 텍스트를 한 번에 밀어주는 용도로 추정.

미해결: \$\$ / \$* 를 실제로 쓰는 맵 화면 예시 미확인(\$? 는 체결그리드로 실측 확인됨). _anmH / anmK * 상수 전체 목록, USRDIR/{pushN} 파일의 실제 포맷과 용도, 이 채널이 1절의 msgK_ARM (0x92)/ msgK_AUX (0x93, 둘 다 미조사 상태였음)와 같은 것인지 다른 것인지 미확인 — **내일 최우선 확인 대상.**

8.12. Wizard 자체 OOP 왕복 — GetData00P / GetData00P2 (요청) + SetData00P (응답) (2026-08-18 확인)

배경 — 8.10절(IBKSCConnector)과는 별개인, 두 번째 OOP 구현

8.10절은 HTS_OpenAPI(운영)/IBKSCConnector (별도 OCX, Wizard/ 바깥)가 OOP TR 요청을 조립하는 문법이었다. 이번에 확인한 건 Wizard/Stream.cpp 안에 있는, IBKSCConnector와 완전히 별개인 Wizard 자체의 OOP 요청/응답 구현이다 — GetData00P (Stream.cpp:822)/ GetData00P2 (Stream.cpp:1092)가 요청을, SetData00P (Stream.cpp:1900)가 응답을 담당한다. 즉 이 프로토콜을 말하는 클라이언트가 최소 두 종류(IBKSCConnector, Wizard 자신) 있다는 뜻이고, 둘 다 서버 입장에서는 같은 us_00P 계열 TR을 주고받는 것으로 보이나 클라이언트 쪽 조립 코드는 중복 구현이다.

어떤 .map 이 이 경로를 타는가: msgK 는 여전히 msgK_AXIS (0x20) 그대로다 — 별도 msgK 값이 아니라, 맵 빌드 옵션 OP_00P / OP_00P2 (h/mapform.h:124,140)가 켜진 화면만 MakeStream 이 GetData00P / GetData00P2 로 요청을 만들고, 그 요청에 axisH->auxs |= auxs00P 비트를 실어 보낸다 (Stream.cpp:1436-1443). 수신측은 OutStream 의 msgK_AXIS 분기에서 이 auxs00P 비트만 보고 SetData00P vs SetDataNRM / SetDataNRM2 로 갈라진다(Stream.cpp:164-165):

```
case msgK_AXIS:
    if (axisH->auxs & auxs00P) // symbol data
        SetData00P(screen, datB, datL);
    else { ... SetDataNRM/NRM2 ... }
```


핵심 발견 — 요청과 응답이 같은 순서표(m_ioR)를 공유해서 정렬을 보장한다 (사용자 가설 확인+정정)

질문하신 가설("송신할 때 탭으로 붙여서 올린 심볼들의 값을 그 순서에 맞게 다시 탭으로 붙여서 내려주지 않았을까")은 골격은 정확히 맞고, 다만 그 "순서"가 어디서 오는지는 조금 다르다. 클라이언트가 요청을 만들 때 즉석에서 정한 순서가 아니라, .map 빌드 시점에 이미 고정된 필드 순서표 screen->m_ioR[] 를, 요청 조립(GetDataOOP)과 응답 파싱(SetDataOOP) 양쪽이 똑같이 for (ii = 0; ii < screen->m_ioL; ii++) 루프로 순회한다(둘 다 이 정확한 형태의 루프를 씀, Stream.cpp:837/1107/1915 대조 확인). 서버는 요청에서 "이 이름의 필드를 원한다"는 선언만 보고, 응답 페이로드에는 필드 이름 없이 값만 탭으로 나열해서 돌려준다 — 클라이언트가 그 값을 다시 같은 m_ioR 순서로 하나씩 소비하며 제자리를 찾는 구조다. 즉 "정렬이 맞는 이유"는 서버가 순서를 지켜서 보내주기 때문이 아니라(그건 전제일 뿐), 클라이언트 쪽 요청 조립과 응답 파싱이 같은 순서표를 공유하도록 설계되어 있기 때문이다 — 이 순서표(.map 의 필드 정의 순서)가 새 플랫폼에서도 요청/응답 양쪽에 반드시 동일하게 보존돼야 한다는 뜻.

요청 조립 — GetDataOOP (OOP1)/ GetDataOOP2 (OOP2)

m_ioR[] 순회 중 필드 종류(kind)/입출력속성(iok)별로 텍스트를 이어붙인다. OOP1과 OOP2는 필드명-값 구분자만 다르고 골격은 동일:

필드 종류	iok	조립 형태 (OOP1, FS = 0x7f)	조립 형태 (OOP2, iFS = '=' / uFS = '@')
FM_EDIT / FM_OUT / 일반	EIO_INPUT	이름 + FS + 값 + \t	이름 + iFS (=) + 값 + \t
"	EIO_INOUT	이름 + FS + 값 + \t 뒤에 이름 + \t 한 번 더(값요청도 겸함)	이름 + uFS (@) + 값 + \t 뒤에 이름 + \t 한 번 더
"	EIO_OUTPUT	이름 + \t (값 없이 이름만 — "이 필드 값을 응답으로 달라"는 선언)	동일
FM_MEMO / FM_BROWSER	위와 동일하되 값 앞에 %05d 형식 5자리 길이 프리픽스 (L_FILED) 추가		
FM_TABLE	-	컬럼명들을 \t 로만 나열(값 없음, 마커도 없음)	
FM_GRID	-	\$ + 이름 + FS + GetEnum(...,99) 결과(연속조회 상태 블롭, 아래 참고) + 컬럼별 이름 (+INPUT/INOUT이면 행수만큼 FS +값 반복)+ \n ... + \t	이름 + \$ (마커 위치가 이름 뒤로 바뀜) + 나머지는 유사
FM_CONTROL (FA_ENUM)	-	\$ + 이름 + FS + 값 + \t	동일
FM_OBJECT	-	재귀 호출(서브맵도 같은 규칙)	동일

8.10절(IBKSCconnector)과 겹치는 부분: 필드명-값 구분자 FS = 0x7f 가 IBKSCconnector가 쓰던 \x7f (DEL)와 정확히 같은 값이고(Stream.h:13), 그리드 마커도 똑같이 \$ 를 쓴다 — 서버가 기대하는 와이어 바이트 자체는 두 클라이언트 구현이 동일한 계약을 따르고 있다는 근거. 다만 IBKSCconnector는 그리드 페이지징 정보를 별도의 이진 구조체(grid_i , 8.10절)로 붙이는 반면,

Wizard 자체 구현은 그런 고정 구조체가 안 보이고 `form->GetEnum(text, formL, 99)` 이라는, 폼 컨트롤 자체에 저장된 opaque 블록(추정: 연속조회/스크롤 상태 — 정확한 내부 포맷은 `CfmGrid::GetEnum / SetEnum` 쪽 소스라 이번 조사 범위 밖, 미확인)을 그대로 읽어 붙인다는 점이 다르다.

응답 파싱 — setDataOOP (Stream.cpp:1900)

setDataH (8.7절, 원장 헤더)로 시작하는 것까지 setDataNRM 과 동일. 이후 `m_ioR[]` 를 순회하며 **필드 종류별로 다른 규칙**:

필드 종류	파싱 방식
FM_EDIT / FM_COMBO / FM_MEMO / FM_BUTTON / FM_BROWSER / FM_OUT	탭(\t) 구분 — <code>text.Find('\t')</code> 까지를 그 필드의 값으로 소비. <code>iok</code> 가 <code>EIO_OUTPUT / EIO_INOUT</code> 인 필드만 값을 받고 (<code>EIO_INPUT</code> 은 건너뛴), <code>FA_SKIP</code> 필드도 건너뛴
FM_GRID	<code>form->GetEnum(dats, pos, 98) / SetEnum(text, pos, 99)</code> 로 opaque 블록을 먼저 소비(요청 때 보낸 98/99 슬롯과 대칭 — 정확한 의미는 미확인) → 이후 데이터는 OOP2면 gFS (0x1b, ESC) 한 바이트, OOP1이면 "\r\t" 두 바이트 를 찾아 그 앞까지를 그리드 블록으로 통째로 <code>form->WriteAll(text)</code> 에 전달 — SetCells / SetTable 을 전혀 안 탐 (8.6절과 무관한, 그리드 컨트롤 자체가 내부 파싱하는 완전히 다른 경로)
FM_TABLE	<code>screen->m_cellR[...]</code> 의 컬럼 정의를 순회하며 <code>\t</code> 구분으로 값을 모아 <code>form->WriteAll(dats)</code>
FM_CONTROL (FA_ENUM)	FM_GRID 와 동일하게 <code>gFS / "\r\t"</code> 구분자 사용, 아니면 일반 필드처럼 <code>\t</code> 구분
FM_OBJECT	재귀 호출(서브맵)

OOP1 vs OOP2 요약(전 구간):

구분자 역할	OOP1 (OP_OOP 만)	OOP2 (OP_OOP2)
요청 필드명-값 (INPUT)	FS = 0x7f	iFS = '='
요청 필드명-값 (INOUT)	FS = 0x7f	uFS = '@'
그리드 마커 위치	\$이름 (마커가 앞)	이름\$ (마커가 뒤)
응답 그리드/ENUM 블록 종결자	"\r\t" (2바이트)	gFS = 0x1b (1바이트)

새 플랫폼 설계 시사점:

- 새 플랫폼은 `.map` 빌드 옵션(`OP_OOP / OP_OOP2`)을 반드시 이관해야 하고, 요청 조립기와 응답 파서가 **동일한 필드 순서표**를 참조하도록 설계해야 한다 — 순서표가 요청/응답 양쪽에서 어긋나면 값이 다른 필드로 밀려 들어가는, IB204200 그리드 오염 사례(고정폭 `SetCells` , 8.6절)와 같은 유형의 정렬 버그가 OOP 경로에서도 똑같이 발생할 수 있다.
- OOP1/OOP2는 **바이트 값 하나(구분자 문자)가 프로토콜 버전을 가르는 유일한 차이**다 — 새 플랫폼은 이 두 프로토콜을 완전히 별개로 재구현하기보다, 구분자 상수 4개(`FS / iFS / uFS / gFS`)를 옵션값으로 뽑아내는 하나의 파서로 통합하는 게 자연스럽다.

3. `GetEnum(...,98) / SetEnum(...,99)` 슬롯이 정확히 뭘 담는지(추정: 그리드 연속조회 상태, IBKSCconnector의 `grid_i.save` 와 유사한 역할일 가능성)는 이번 조사로 확정하지 못했다 — **다음 조사 대상**(9절에 반영).

9. 다음 조사 대상 (미완료)

- **ParseSCC / SetCC 의 CC_* 플래그 전체 목록** — `CC_PRO / CC_MAND / CC_SEND / CC_VIS / CC_ENB / CC_SET` 등 확인됨 (`Stream.cpp:2900-2928`), 전체 목록과 각각의 정확한 UI 효과는 추가 확인 여지
- ~~`SetData00P`~~ — **확인 완료(2026-08-18), 8.12절 참고.** Wizard 자체의 OOP 요청(`GetData00P / GetData00P2`)+응답(`SetData00P`) 왕복 전체를 `Stream.cpp` 안에서 확정 — 8.10절(IBKSCconnector)과는 별개 구현이며, 필드 순서 정렬은 양쪽이 공유하는 `m_ioR[]` 순회로 보장됨을 확인
- **`SetDataNRM2 / SetDataTAB / SetDataTAB2`** — `SetDataNRM / SetData00P` 외 나머지 포맷 변형 상세 미조사
- **(신규, 8.12절) `CfmGrid::GetEnum / SetEnum` 의 슬롯 98/99가 담는 opaque 블록의 정확한 의미** — OOP 요청/응답 양쪽에서 그리드 연속조회 상태로 추정되나 `fmGrid.cpp` 쪽 소스 확인 필요
- **`SetDataH`** — 레코드 헤더 파싱 상세 미조사
- **`SetTable` 엔 있고 `SetCells` 엔 없는 FCC/RCC/SCC 처리** — 왜 `GO_TABLE` 방식만 셀별 동적 속성제어를 지원하는지 설계 의도 미확인
- **`WM_USER` 커스텀 메시지의 정확한 용도**
- **TR 요청(사용자가 조회 버튼 누르는 것) → 소켓 송신 경로** — **확인 완료(2026-07-30), 8.8절 참고.** `RouteTR` 이 `CGuard::Write(char*, int, bool)` 로 최종 소켓 전송하며, 한 번의 write에 여러 화면(unit)의 `_axisH` 프레임이 배치로 묶일 수 있음
- **`CD11::OnAxis` (DLL 기반 작업영역)의 실제 파싱 로직** — **`cclient` 와 다르다는 것 자체는 확인됨(2026-07-31):** `CStream::OutStream / SetDataNRM` 을 전혀 안 타고 받은 바이트를 그대로 `WM_USER` 로 로드된 DLL에 던진다(`D11.cpp:530`, [`CD11-OnAxis-raw`] 로그 추가, @docs/DebugLogGuide.md 7절). 다만 그 DLL 내부의 실제 파싱 로직 자체는 Wizard 소스 밖이라 여전히 미조사 — 실사용 사례: 9524 (이벤트 데이터 조회, 기획부) 화면이 이 경로를 탐
- **(신규, 2026-08-17, 8.11.3절) `$( m_push )` 전용 채널 `_anmH / FEV_PUSH`** — 구조(요청 `SetPush`/수신 `OnPush`/원형버퍼 반영)는 코드로 확정됐으나: (1) `anmK_*` 상수 전체 목록 미확인(`anmK_ALIVE / anmK_PUSH` 만 확인됨), (2) `USRDIR/{pushN}` 로컬 파일의 실제 포맷/용도 미확인, (3) 1절의 `msgK_ARM (0x92)/ msgK_AUX (0x93)`와 이 `_anmH` 채널이 같은 것인지 완전히 별개인지 미확인, (4) `$$ / $*` 를 실제로 쓰는 맵 화면 예시 미확인(`$?` 만 체결그리드로 실측됨) — **내일 최우선 조사 대상**

10. 관련 문서

- @docs/WizardArchitecture.md — 클래스 계층, 7절에 전체 클래스 레퍼런스
- @docs/RealtimeCodeIndex_Investigation.md — RTM 중목코드 매칭 상세
- @docs/AxisformArchitecture.md — 컨트롤(`CfmBase` 24종) 렌더링 레이어 상세
- @docs/KnowledgeBase.md — 트러블슈팅/설계의도 누적 기록